# Group Project Proposal

Interruptible MLE Agents: Human Intervention in Long-Horizon Tree Search

Group ID: _____

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

## Administrative Information

| | |
|---|---|
| **Group members** | Nabil Rakaiza Abror (A0000000X)<br>Name (Student ID)<br>Name (Student ID)<br>Name (Student ID) |
| **Project direction** | Human-in-the-loop, tool-using agents for machine learning engineering. |
| **Repository** | https://github.com/nabilrakaiza/interruptible-mle-agent (private during the semester; access granted to the instructor on request) |

## Project Summary

Machine learning engineering (MLE) agents run autonomously for many hours, steering their search with a validation score they compute themselves. Recent work shows that this score drifts away from true held-out performance as the run proceeds: selecting the final submission by test score rather than validation score would raise the medal rate by 9 to 13 percentage points, and by 16.6 points for greedy agents [4]. The agent cannot detect this from inside its own loop, because the signal it would need is exactly the signal it does not have. A human watching the search could.

We propose an MLE agent built on tree search over candidate solutions that is *interruptible*: it exposes its search tree, current plan, and distilled findings in real time, pauses at well-defined safe points, and accepts a small set of structured human interventions before resuming. The agent is implemented as a small team of specialised sub-agents with explicit tool permissions, each of which doubles as an intervention surface. We evaluate on a subset of MLE-bench Lite [1], comparing autonomous runs, scripted intervention policies that simulate a monitoring human, and a small number of live human sessions. We measure outcome quality, the validation–test gap, interaction cost, and monetary cost. The project aims to produce the first characterisation of which interventions into an MLE agent are worth a human's attention, and at what cost.

## 1. Problem, Research Question, and Motivation

### 1.1 Research question

> **Which human interventions into a tree-search MLE agent improve its outcomes, and at what interaction cost?**

Two sub-questions follow. First, *can a human, given a progress view of the search, detect and correct the validation–test divergence that the agent cannot see?* Second, *which kind of intervention matters*: those that change the agent's next *action* (redirecting the search, pruning a branch, overriding the final submission), or those that change the agent's *beliefs* (injecting domain knowledge, correcting a stored conclusion) and leave the decision to the agent?

### 1.2 Motivation

The failure we target is measured, large, and currently unaddressed.

**The loss is quantified.** Toledo et al. [4] tracked what an agent believes it is achieving (mean 5-fold cross-validation score) against what it actually achieves on the held-out test set across a 24-hour run. The believed score keeps climbing while the true score plateaus and eventually declines. Choosing the final node by test score instead of validation score would raise the medal rate by 9 to 13 absolute points depending on search policy, and oracle final-node selection alone eliminates the entire gap for agents using Monte Carlo tree search or evolutionary search. Their mitigation, submitting the top-$k$ nodes ranked by validation score, recovers roughly 10% of performance at $k = 3$, which is real but is a blind hedge rather than a judgement.

**The failure is invisible from inside the loop.** Nam et al. [5] report a case on `spaceship-titanic` in which validation accuracy rose from 0.819 to 0.868 while test accuracy fell from 0.803 to 0.734, caused by feature engineering that used the target variable. The agent experienced this as an improvement. Yamada et al. [3] audited their own workshop-accepted paper after the fact and found roughly 57% overlap between the training and test sets in the generated code, along with a figure contradicting the claim it was cited for. Three human reviewers did not catch either.

**No existing MLE agent admits a human.** AIDE [2], AIRA [4], MLE-STAR [5], ML-Master [6] and ML-Master 2.0 [7] are autonomous by construction: each decides internally when to stop debugging, which branch to pursue, and what to submit. MLE-STAR notes that a human can supply a model description or override the target code block, but this is neither developed nor evaluated. Human-in-the-loop mechanisms for general LLM agents exist, such as rollback and message editing in AGDebugger [9] and co-planning in Magentic-UI [10], but the natural unit of intervention in an MLE search is a scored node in a solution tree, not a turn in a dialogue.

### 1.3 Why an agent-based approach

The task is open-ended code generation from a task description and a raw dataset. Classical AutoML and neural architecture search require a predefined search space, which is unavailable here; the LLM-agent framing exists because the search space is the space of programs [2]. The system is agentic in the sense studied in this course: it interleaves reasoning with actions and environment feedback [11], searches over candidate solutions with a verifier [12], acts through a constrained agent–computer interface [13], and maintains reflective memory across a long horizon [14]. Our question is specifically about *agent* autonomy: it arises only because the system takes hundreds of consequential decisions without supervision.

### 1.4 What success means

We will consider the project successful if we deliver:

1. a working interruptible MLE agent that produces valid submissions on the majority of our task subset;
2. a documented intervention vocabulary with defined safe-point semantics, organised along the action/belief axis of Section 1.1;
3. a controlled comparison of autonomous versus intervened runs on outcome, process, interaction-cost, and monetary-cost metrics; and
4. a reproducible artefact: code, configurations, saved search trees, and intervention logs sufficient to regenerate every table in the report.

Beating the current state of the art on MLE-bench (56.4% medal rate [7]) is explicitly *not* a goal; that figure was obtained under compute budgets far beyond ours. A negative result, in which interventions do not improve outcomes, is informative and will be reported as such.

## 2. Related Work and Proposed Contribution

### 2.1 Benchmark

MLE-bench [1] comprises 75 offline Kaggle competitions with local grading code and human baselines from the original leaderboards. The headline metric is the percentage of competitions in which the agent earns any medal. We use MLE-bench Lite, the 22 low-complexity competitions, which is the subset most published agent work reports on and the only tier feasible on student compute.

### 2.2 MLE agents

AIDE [2] reframes MLE as optimisation over the space of Python scripts. Each node in a search tree is a self-contained script with a scalar fitness, so the prompt never accumulates the full interaction history. A greedy policy repeatedly applies one of three operators (draft, debug, improve) to the best working solution.

AIRA-dojo [4] formalises such agents as a tuple of fitness function, selection policy, operator set, operator policy, and termination rule, which allows components to be varied independently. Two findings matter for us. First, the operator set, not the search policy, is the bottleneck: replacing greedy search with MCTS or evolutionary search while holding AIDE's operators fixed produced no gain, and ablating AIDE's memory operator changed almost nothing. Second, agents systematically overfit their proxy metric (Section 1.2).

MLE-STAR [5] retrieves candidate models from the web to form an initial solution, then uses generated ablation studies to identify the highest-impact code block and refines only that block. It also introduces explicit data-leakage and data-usage checkers, and reaches 43.9% on MLE-bench Lite with a small, inexpensive model.

ML-Master [6] runs MCTS with parallel branches and supplies the reasoning model only with its parent and sibling nodes. ML-Master 2.0 [7] generalises this into Hierarchical Cognitive Caching,

promoting raw traces into distilled judgements at phase boundaries and into transferable wisdom at task completion. It is the current state of the art at 56.4%. Its distilled-judgement tier is close in form to the reflections of Generative Agents [14], and we adopt a similar format for our progress view.

FML-bench [8] argues that these benchmarks measure only final performance and proposes an exploration-diversity metric over an agent's proposals.

### 2.3 Human-in-the-loop agents

AGDebugger [9] shows that rollback and message editing mitigate cascading errors in long-horizon multi-agent systems. Magentic-UI [10] supports co-planning, action approval, and answer verification for general digital tasks. Neither addresses a search over machine learning solutions, where interruption must respect long-running training jobs and where the artefact under review is a scored program.

### 2.4 Proposed contribution

The tree representation and the draft/debug/improve operators are adapted from AIDE [2] with the prompt-level improvements reported by AIRA-dojo [4]; the progress summary format is adapted from ML-Master 2.0 [7]; the leakage checker follows MLE-STAR [5]. Our contribution is:

1. **Interruptibility as a property of the search loop.** A checkpoint and resume mechanism with defined safe points, a structured intervention vocabulary, and a real-time progress view. No published MLE agent provides this.
2. **The first measurement of intervention value in MLE.** A controlled comparison of autonomous runs, scripted intervention policies, and live human sessions, reported against outcome metrics, the validation–test gap, and interaction cost.
3. **Structural separation of evaluation from optimisation.** The evaluation harness is built once by a dedicated sub-agent and is immutable to the agent that writes solution code, enforced by file permissions rather than by instruction. Prior systems check for leakage after the fact [5]; we make the contract structural, stress-test the checker, and ablate it.

## 3. Agent and Task Setup

### 3.1 Task and environment

Given a competition description and a dataset directory, the system must produce `submission.csv`. Submissions are graded locally by MLE-bench's grading code against the competition's medal thresholds. Each run executes inside an isolated container with a fixed CPU, memory, and GPU quota, a per-run wall-clock budget, and a per-script execution timeout.

### 3.2 Sub-agents

- **A1, Evaluation agent.** Runs once at the start. Inspects the data, constructs three disjoint splits (training, validation, and a small *probe* split), and writes the scoring function to a harness

module. The validation split drives node selection. The probe split is hidden from A3 and from selection and is shown only in the progress view, where it gives the monitor a signal the agent never optimises against. A1 also screens each candidate script for target leakage and unused data sources before execution, following [5].

- **A2, Retrieval agent.** On draft operations, queries the web for candidate model families and example usage for the task type, and returns them as model cards in a fixed schema.
- **A3, Coding agent.** Implements the draft, debug, and improve operators. Produces a short plan plus a self-contained script that trains, scores itself through the harness, and writes a submission.
- **A4, Reflection agent.** At phase boundaries, compresses the traces of completed nodes into short findings, each with the evidence that produced it (node identifiers and scores), and removes the raw traces from working context.
- **A5, Tree controller.** Maintains the search graph, selects the next node and operator, enforces stopping rules, and owns checkpointing and intervention application. It is the only writer to the tree.

### 3.3 Tool permissions

Permissions are enforced by the environment, not by prompts. Table 1 lists what each component may do.

| | **Read data** | **Write harness** | **Execute code** | **Web access** | **Write tree / KB** |
|---|---|---|---|---|---|
| A1 Evaluation | yes | yes (once) | yes, sandbox | no | no |
| A2 Retrieval | no | no | no | allowlisted | model cards only |
| A3 Coding | yes | **no** | yes, sandbox | no | no |
| A4 Reflection | no | no | no | no | findings only |
| A5 Controller | no | no | no | no | yes |
| Human monitor | view | no | no | no | via queue only |

Table 1: Permission matrix. A3 cannot modify the harness or reach the network, so it can neither redefine its objective nor fetch arbitrary code.

### 3.4 Knowledge base

A structured store, not a vector index in the first version. It holds model cards from A2 and findings from A4, each with its source or evidence. AIRA-dojo [4] found that removing AIDE's memory operator barely changed performance, while ML-Master 2.0 [7] found its knowledge tier necessary for top results. The contradiction is unresolved, so we treat memory as a hypothesis to test. Retrieval over the store will be introduced only if injected context is observed to saturate, since within a single run the number of findings is small.

### 3.5 Interruption semantics

**Safe points** are node boundaries: after a script has executed and its result has been scored and written to the tree, and before A5 selects the next node. This is the only moment at which all state

is consistent. At a safe point A5 serialises the tree, knowledge base, harness, elapsed budget, and random state, then drains an intervention queue. The human interface is a Streamlit dashboard running as a separate process from the agent: it reads the checkpoint and appends to the queue, and never touches live state. The agent is therefore the only writer to the tree, and the dashboard can refresh, crash, or be opened by several users without affecting the run. Scripted intervention policies (Section 4.2) use the same queue, so human and scripted interventions pass through an identical code path. Every checkpoint is retained, which allows a finished run to be replayed step by step for analysis and demonstration.

**Intervention vocabulary**, grouped by what it changes:

- *Action interventions.* **Prune** a node or subtree; **Redirect** the next node and operator; **Override** the final submission.
- *Belief interventions.* **Inject** a model card or approach into the knowledge base; **Correct** or delete a finding from A4.
- *Budget intervention.* **Rebudget** the remaining wall clock.

A running script is not interrupted by default. As an optional budget-saving action, the monitor may terminate a running script; the node is then marked *abandoned* and contributes no score, since a partial score is not comparable to a complete one and would corrupt selection. Wall clock spent on an abandoned node is counted as part of that intervention's cost. The budget clock does not pause while the agent waits on a human, so intervened and autonomous runs receive equal time.

Queued actions are applied in timestamp order. An action whose target no longer exists or is no longer in the expected state is dropped and the drop is logged. Every applied or dropped action is recorded with its timestamp, target, and optional note, which yields a complete audit trail of human influence on each run.

### 3.6 Constraints and stopping conditions

A2 may fetch model documentation, library examples, and research papers from an allowlist, and is prohibited from retrieving competition solutions, winning notebooks, or leaderboard discussion, in line with MLE-bench's rule against looking up solutions [1]. Search terminates on wall-clock exhaustion or a node cap. A debug chain terminates after a fixed number of consecutive failures, and an improve chain terminates after a fixed number of attempts that fail to exceed a small relative-improvement threshold, following [6]. Failed nodes remain in the tree, marked buggy, and are visible in the progress view.

## 4. Proposed System or Study Design

### 4.1 Main loop

1. A1 builds the train, validation, and probe splits and the harness, then freezes the harness.
2. A5 selects a node and an operator (greedy on validation score, with a small probability of revisiting a buggy node, following [2]).

3. On a draft, A2 supplies model cards; on any operator, the knowledge base supplies current findings.
4. A3 writes a plan and a script. A1 screens it for leakage and unused data.
5. The script executes in the sandbox and is scored on validation and probe. It also writes a test-set submission, which is graded but never shown to any agent. The result becomes a new node.
6. A4 records findings if the node closes a phase.
7. **Safe point.** A5 checkpoints and applies queued interventions, then returns to step 2.
8. On termination, the final submission is chosen by the configured policy: best validation, top-$k$, or human override.

The part owned by the group is the checkpoint, queue, and progress-view layer, the evaluation contract between A1 and A3, and the study design. The final report will include a system diagram and annotated example trajectories.

### 4.2 Comparisons and ablations

Table 2 lists the conditions. C2 and C3 require *no additional runs*: every node retains its script, validation score, and graded test score, so both are computed post hoc from the saved trees of C1. They bound how much any selection-time intervention could be worth and reproduce the analysis of [4] on our own system.

Live human sessions (C6) are not used as replayable data. LLM outputs are nondeterministic, so a recorded action such as "prune node 17" refers to a different script on any rerun. Instead, we observe what participants do and why, express the observed behaviour as explicit rules, and evaluate those rules as scripted conditions (C4, C5) over fresh seeds. The live sessions therefore serve as a source of hypotheses and qualitative evidence.

| ID | Condition | Purpose | Runs |
|---|---|---|---|
| C1 | Autonomous baseline | Greedy search, best-validation submission. | yes |
| C2 | Oracle final-node selection | Upper bound on selection-time gain. | post hoc |
| C3 | Top-$k$ submission ($k = 3$) | Blind-hedge comparison from [4]. | post hoc |
| C4 | Scripted: divergence pruning | Prune a subtree when validation gains are not matched on the probe split. Action intervention. | yes |
| C5 | Scripted: finding correction | Delete any finding whose supporting nodes are later contradicted. Belief intervention. | yes |
| C6 | Live human sessions | Qualitative runs with think-aloud; source of C4/C5 rules. | few |
| A1-off | No evaluation agent | A3 writes its own split and metric. Tests the structural separation claim. | yes |
| A4-off | No reflection agent | Tests whether accumulated findings help at all. | yes |
| A2-off | No retrieval agent | Tests web-retrieved models. Optional. | if budget |

Table 2: Experimental conditions. One component changes per row, following the attribution argument of [4]. C4 and C5 cover one action and one belief intervention respectively.

# 5. Preliminary Evaluation Plan

## 5.1 Metrics

**Outcome**, reported as in MLE-bench [1]: valid submission rate, above-median rate, any-medal rate.

**Process**, where our contribution is visible:

- **Validation–test gap**: medal rate when the final node is selected on test score minus medal rate when selected on validation score, per condition. This is the quantity the project exists to move.
- **Gap trajectory**: the same quantity over wall-clock time, testing whether divergence widens as reported in [4].
- **Probe fidelity**: correlation between probe and test scores across nodes, which determines whether the monitor's signal is trustworthy.
- **Exploration diversity** [8], to test whether intervention broadens or narrows the search.

**Interaction cost**: number of interventions per run, their timing, human time spent per intervention in C6, and wall clock lost to abandoned nodes.

**Monetary and system cost**: input and output tokens per node and per run, API cost, wall clock, and GPU-hours, logged for every condition.

## 5.2 Reliability checks on LLM components

A1's leakage screen and A4's findings are both LLM judgements, which can be unreliable [15]. We therefore run two targeted checks:

- **Leakage stress test.** We construct a set of scripts with planted leakage of known types (target used as a feature, test statistics used in imputation, validation rows in training) alongside clean controls, and report A1's detection rate and false-positive rate.
- **Finding accuracy.** For a sample of A4 findings, we check whether the claim is supported by the cited nodes' scores, and report the fraction that is unsupported.

## 5.3 Protocol

**Task subset.** Six MLE-bench Lite competitions chosen for small dataset size and modality coverage: `spooky-author-identification` and `detecting-insults-in-social-commentary` (text), `random-acts-of-pizza` and `nomad2018-predict-transparent-conductors` (tabular), `aerial-cactus-identification` and `leaf-classification` (image). All are under 40 MB.

**Budget.** A 4-hour wall-clock budget per run instead of MLE-bench's 24 hours, trading absolute performance for seed count, following the argument in [4] that a smaller task set with more seeds yields more reliable conclusions.

**Seeds.** Five seeds per task for C1, C4, C5; three for the ablations. AIRA-dojo [4] shows that agent rankings can flip under three and even five seeds and recommends at least ten. We cannot afford ten, so we report bootstrapped 95% confidence intervals and do not claim rankings whose intervals overlap.

**Reproducibility.** LLM outputs are not deterministic even at temperature zero, owing to provider-side batching and non-associative floating-point reduction, and generated training scripts carry their own nondeterminism. We therefore make distributional rather than exact-replay claims. We pin model version identifiers, fix and record seeds for every component we control, and log every prompt, raw completion, model version, sampling parameter, and timestamp.

**Error analysis.** Failed and poor nodes will be coded into a taxonomy (runtime error, timeout, invalid submission format, leakage, overfitting to validation, unused data, stagnation), reported per condition. We will include at least two annotated trajectories, one where intervention helped and one where it did not.

### 5.4 Mapping claims to evidence

| Claim | Evidence |
|---|---|
| The agent overfits its own proxy metric | Gap trajectory over C1 runs |
| The probe gives the monitor a usable signal | Probe fidelity across C1 nodes |
| A human-style policy can correct the drift | Gap and medal rate, C4 versus C1, bounded above by C2 |
| Judgement beats a blind hedge | C4 versus C3 |
| Action and belief interventions differ in value | C4 versus C5 |
| Structural evaluation separation matters | Leakage incidents and gap, A1-off versus C1; stress-test detection rate |
| Accumulated findings help, or do not | Medal rate, A4-off versus C1; finding accuracy |
| Interventions are worth their cost | Medal rate change against interaction cost and lost wall clock |

Table 3: Each claim is tied to a specific comparison.

## 6. Risks, Limitations, and Responsible Practice

**Prompt injection through retrieval.** A2 reads untrusted web content, and text derived from it reaches A3, whose output is *executed*. A malicious or merely misleading page could therefore steer generated code. Mitigations follow the instruction-hierarchy principle [16] that retrieved content is data, not instructions: A2 returns only a fixed model-card schema rather than free text; model cards are delimited and labelled as untrusted in A3's prompt; A2's sources are allowlisted; and the sandbox has no network access, no credentials, and no access to the host, so a compromised script is contained. We will red-team this by inserting a page with an embedded instruction into A2's results and reporting whether it propagates.

**Compute.** A full MLE-bench evaluation costs roughly 1,800 GPU-hours [1]. Our plan (Section 7) is far smaller, runs on local machines and free-tier cloud credits, and all post hoc conditions add no compute. If the budget proves infeasible, we drop tasks before seeds, since seed count is what protects our conclusions, and drop A2-off first.

**Reduced budget changes absolute numbers.** AIRA-dojo [4] observed that meaningful divergence between agents appears only after roughly 15 hours. Our 4-hour runs will report lower medal rates than published figures, and our conclusions concern relative differences between con-

ditions on a shortened horizon.

**Scripted policies may not represent real users.** C4 and C5 are reproducible but not behaviourally faithful. C6 mitigates this only partially; with a handful of participants, drawn from the group, it is illustrative and will not be presented as a user study.

**Probe split costs data.** Carving a probe split reduces training data slightly. It is applied identically in all conditions so comparisons remain fair, but absolute scores are affected.

**Reflection may mislead.** A finding written from one noisy run can propagate silently. Each finding stores its evidence, C5 tests correction, and A4-off bounds the damage. A null result resolves a genuine contradiction in the literature and will be reported as a finding.

**Contamination.** MLE-bench competitions are public and may appear in pretraining data. The original authors found no correlation between model familiarity and performance [1]. We additionally restrict A2's sources and will spot-check final solutions against public discussion as in [5].

**Conduct.** Agents execute generated code only inside an isolated container. API keys are held in environment variables and excluded from the repository. We use only publicly released MLE-bench data and will not redistribute competition datasets. Participants in C6 are group members; no external personal data is collected.

# 7. Required Resources and Work Plan

## 7.1 Reused versus built

| **Reused (documented)** | **Purpose** |
|---|---|
| MLE-bench [1] | Tasks, data preparation, grading code, medal thresholds |
| AIDE open-source code [2] | Starting point for operators and tree structure |
| Hosted LLM API | All sub-agents; a low-cost model tier, which MLE-STAR showed is viable [5] |
| Container runtime | Sandboxed execution |
| Streamlit | Progress dashboard and intervention interface |
| Python ML stack | PyTorch, scikit-learn, LightGBM, and similar inside the sandbox |
| **Built by the group** | |
| Evaluation agent and probe split; permission enforcement; checkpoint, queue, and safe-point mechanism; dashboard views and run replay; scripted intervention policies; reflection store with evidence; logging and cost accounting; leakage and prompt-injection stress tests; analysis pipeline. | |

## 7.2 Compute and cost

Run-requiring conditions: C1, C4, C5 at 6 tasks $\times$ 5 seeds (90 runs), and A1-off, A4-off at 6 tasks $\times$ 3 seeds (36 runs), giving 126 runs of up to 4 hours, or at most about 500 machine-hours. In practice this is lower because small-data tasks terminate early.

**Source of compute.** Group members' local machines, supplemented by free-tier cloud compute (for example Modal). The task subset was chosen so that every task is feasible on CPU: the text and tabular tasks need no GPU, `leaf-classification` ships precomputed tabular features,

and `aerial-cactus-identification` uses $32 \times 32$ images small enough for CPU training. A GPU therefore speeds up the two image tasks but is not required. Runs are independent, so CPU runs are spread across members' machines and GPU-accelerated runs are sent to cloud credits. Because free tiers are capped, the plan does not depend on them: if credits run out, the remaining runs execute on CPU, and if total machine time is still insufficient we apply the fallback in Section 6.

**API cost.** The LLM API is expected to cost more than compute. It will be estimated from measured tokens per node in a pilot run during the first build week, and reported with per-condition totals in the final report.

### 7.3 Timeline

| Dates | Milestone |
|---|---|
| 29 Sep – 11 Oct | MLE-bench setup on the six tasks; A3 and A5 baseline; A1 harness and probe split; pilot C1 run for cost estimation. (Quiz on 5 Oct.) |
| 12 Oct – 18 Oct | Checkpoint and queue layer; progress view; A4; permissions. Lighter week (individual assignment due 16 Oct). |
| 19 Oct – 28 Oct | Full C1, C4, C5 and ablation runs; C6 live sessions; stress tests. |
| 29 Oct – 12 Nov | Analysis and presentation. If the group is assigned the earliest slot (29 Oct), the presentation will report pilot and partial results and the full analysis follows in the report. |
| 13 Nov | Final report. |

### 7.4 Tentative roles

- **Member 1, infrastructure and experiments**: MLE-bench setup, sandbox, permissions, compute, and run orchestration.
- **Member 2, core agent**: A3 and A5, operators and search loop.
- **Member 3, evaluation and memory**: A1, probe split, A4, and the leakage and prompt-injection stress tests.
- **Member 4, interaction layer**: checkpointing, queue, progress view, and the C6 sessions.

Statistical analysis, error analysis, and writing are shared. Roles will be assigned to named members, and a contribution statement will accompany the final report as required.

## References


[1] J. S. Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *ICLR*, 2025. arXiv:2410.07095.

[2] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138, 2025.

[3] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066, 2025.

[4] E. Toledo, K. Hambardzumyan, M. Josifoski et al. AI Research Agents for Machine Learning: Search, Exploration, and Generalization in MLE-bench. arXiv:2507.02554, 2025.

[5] J. Nam, J. Yoon, J. Chen, J. Shin, S. Ö. Arık, and T. Pfister. MLE-STAR: Machine Learning Engineering Agent via Search and Targeted Refinement. arXiv:2506.15692, 2025.

[6] Z. Liu, Y. Cai, X. Zhu et al. ML-Master: Towards AI-for-AI via Integration of Exploration and Reasoning. arXiv:2506.16499, 2025.

[7] X. Zhu, Y. Cai, Z. Liu et al. Toward Ultra-Long-Horizon Agentic Science: Cognitive Accumulation for Machine Learning Engineering. arXiv:2601.10402, 2026.

[8] Q. Zou, H. H. Lam, W. Zhao et al. FML-bench: Benchmarking Machine Learning Agents for Scientific Research. arXiv:2510.10472, 2025.

[9] W. Epperson et al. Interactive Debugging and Steering of Multi-Agent AI Systems. *CHI*, 2025.

[10] H. Mozannar et al. Magentic-UI: Towards Human-in-the-loop Agentic Systems. Microsoft Research, 2025.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

[12] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.

[13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS*, 2024.

[14] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.

[15] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.

[16] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024.